

PERTEMUAN 12

VIDEO

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu memahami grafik interaktif dan aplikasi real-time.

Pokok Bahasan:

1. Menampilkan video langsung (live)
2. Menampilkan video yang direkam
3. Membuat cermin perangkat lunak (software mirror)
4. Dasar-dasar computer vision: cara menggunakan kamera video sebagai sensor
5. Soal Latihan

12.1 Live Video

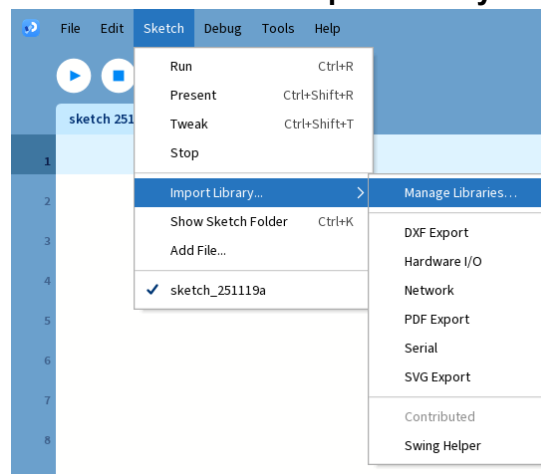
Setelah Anda mempelajari gambar statis di Processing, Anda siap untuk beralih ke gambar bergerak, khususnya dari kamera langsung (dan nanti, dari video yang direkam). Saya akan mulai dengan menjelaskan langkah-langkah dasar untuk mengimpor pustaka video dan menggunakan kelas **Capture** untuk menampilkan video langsung.

Langkah 1. Impor pustaka video Processing.

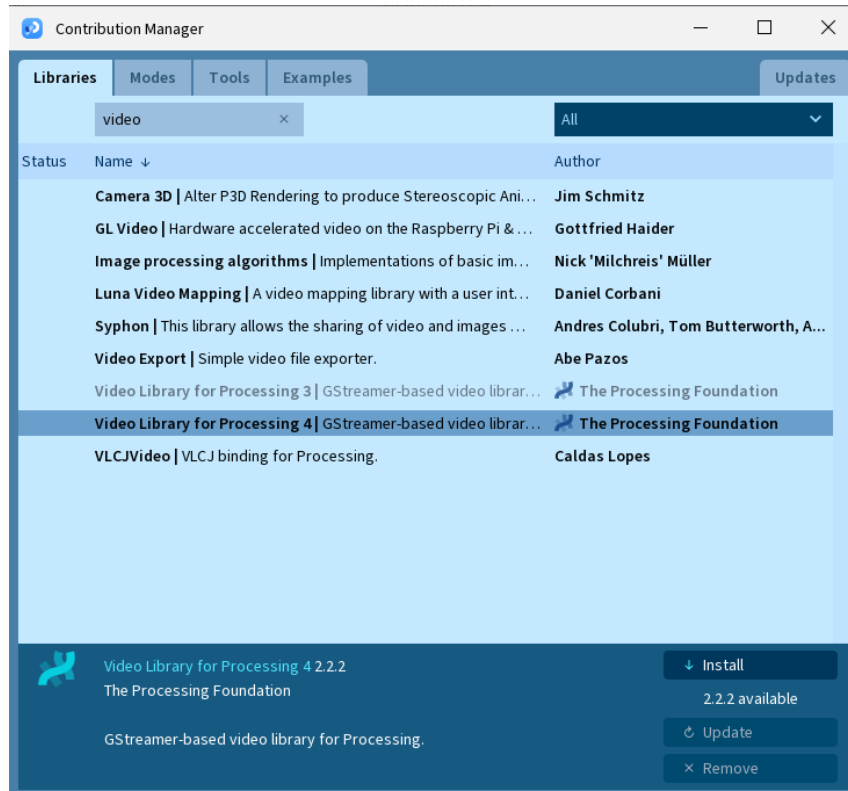
Meskipun pustaka video dikembangkan dan dikelola oleh Processing Foundation, karena ukurannya, pustaka tersebut tetap harus diunduh secara terpisah melalui *contributions manager*.

Dunia pustaka pihak ketiga (juga dikenal sebagai pustaka “kontribusi”) untuk Processing mirip seperti wilayah barat yang liar. Pada saat buku ini ditulis, terdapat 113 pustaka kontribusi yang secara resmi terdaftar di situs Processing, dengan kemampuan yang mencakup mulai dari simulasi fisika hingga *packet sniffing*, *computer vision*, teks generatif, hingga kontrol GUI. Pencarian online kemungkinan akan mengungkap ratusan pustaka lain yang tidak terdaftar (dan mungkin tidak kompatibel dengan versi Processing yang terbaru). Saya akan membahas kode dari beberapa pustaka kontribusi sepanjang sisa buku ini. Untuk saat ini, mari kita fokus pada proses instalasi pustaka itu sendiri.

Proses instalasi pustaka kontribusi sama dengan proses instalasi pustaka Processing Foundation seperti video dan sound. Saya akan menggunakan pustaka video sebagai contoh di bawah ini. Langkah pertama adalah membuka *Processing Contributions Manager*. Anda dapat mengaksesnya melalui menu **Sketch** → **Import Library** → **Manage Libraries**.



Tampilan tersebut akan memperlihatkan **Library Manager**. Di sana, Anda dapat mencari melalui daftar pustaka untuk menemukan pustaka yang Anda butuhkan. Anda dapat memfilter daftar berdasarkan kategori atau dengan mengetikkan kata kunci pada kotak pencarian. Sebagai contoh, saya dapat dengan cepat menemukan pustaka *video* milik Processing Foundation dengan mengetik “**video**”.



Untuk menginstal sebuah pustaka, cukup tekan tombol “**install**” dan tunggu hingga pustaka tersebut selesai diunduh. Itu saja! Meskipun tidak wajib, jika Anda mengalami masalah, cobalah untuk me-restart Processing setelah instalasi. Anda juga dapat memperbarui pustaka melalui *manager* jika suatu pembaruan dirilis. Anda dapat memeriksa pustaka mana saja yang memiliki pembaruan melalui **Library Manager**.

Setelah pustaka terpasang, langkah berikutnya adalah mengimpor pustaka tersebut ke dalam kode Anda. Hal ini dilakukan dengan memilih menu **Sketch** → **Import Library** → **Video**, atau dengan mengetikkan baris kode berikut (yang harus diletakkan di bagian paling atas *sketch* Anda):

```
import processing.video.*;
```

Menggunakan menu “**Import Library**” tidak melakukan apa pun selain secara otomatis memasukkan baris tersebut ke dalam kode Anda, sehingga pengetikan secara manual sepenuhnya setara.

Langkah 2. Deklarasikan sebuah objek *Capture*.

Anda baru saja melihat bagaimana membuat objek dari kelas-kelas yang sudah ada di dalam bahasa Processing, seperti **PShape** dan **PImage**. Perlu dicatat bahwa kedua kelas tersebut

merupakan bagian dari pustaka **processing.core**, sehingga tidak memerlukan *import statement*. Pustaka **processing.video** memiliki dua kelas yang berguna di dalamnya — **Capture** untuk video langsung, dan **Movie** untuk video yang direkam. Pada langkah ini, saya akan mendeklarasikan sebuah objek **Capture**.

```
Capture video;
```

Langkah 3. Inisialisasikan objek **Capture**.

Objek **Capture** bernama “video” sama seperti objek lainnya. Untuk membuat sebuah objek, Anda menggunakan operator **new** diikuti dengan **constructor**-nya. Pada objek **Capture**, kode tersebut biasanya ditulis di dalam fungsi **setup()**.

```
video = new Capture();
```

Baris kode di atas masih belum memiliki argumen yang sesuai untuk konstruktor. Ingat, ini bukan kelas yang Anda tulis sendiri, sehingga tidak ada cara untuk mengetahui apa saja yang diperlukan di dalam tanda kurung tanpa melihat referensi daring (<http://www.processing.org/reference/libraries/video/Capture.html>).

Referensi tersebut akan menunjukkan bahwa ada beberapa cara untuk memanggil konstruktor **Capture**. Cara pemanggilan konstruktor yang umum adalah dengan tiga argumen:

```
void setup() {  
    video = new Capture(this, 320, 240);  
}
```

Mari kita bahas argumen-argumen yang digunakan dalam konstruktor **Capture**.

- **this** — Jika Anda bingung dengan apa arti *this*, Anda tidak sendirian. Ini adalah pertama kalinya *this* muncul dalam contoh apa pun di buku ini sejauh ini. Secara teknis, *this* mengacu pada instance dari sebuah kelas di mana kata *this* muncul. Sayangnya, definisi seperti ini cenderung membuat kepala pusing. Cara yang lebih mudah untuk memahaminya adalah sebagai pernyataan yang merujuk pada diri sendiri. Lagipula, bagaimana jika Anda perlu merujuk program Processing Anda sendiri dari dalam kode Anda? Anda mungkin mencoba mengatakan “saya” atau “aku.” Namun, kata-kata tersebut tidak tersedia dalam Java, sehingga Anda menggunakan *this*. Alasan Anda memasukkan *this* ke dalam objek **Capture** adalah bahwa Anda memberi tahu objek tersebut:
“Hei, dengarkan, saya ingin melakukan *video capture*, dan ketika kamera memiliki gambar baru, saya ingin kamu memberi tahu *sketch* ini.”
- **320** — Untungnya, argumen pertama, *this*, adalah satu-satunya yang membingungkan. Angka 320 mengacu pada lebar video yang ditangkap oleh kamera.
- **240** — Tinggi dari video.

Namun, ada beberapa kasus di mana cara di atas tidak dapat digunakan. Misalnya, bagaimana jika Anda memiliki beberapa kamera yang terhubung ke komputer Anda? Bagaimana Anda memilih kamera yang ingin Anda gunakan untuk menangkap video? Selain itu, dalam beberapa kasus yang jarang terjadi, Anda mungkin juga ingin menentukan *frame rate* dari kamera. Untuk kasus-kasus tersebut, Processing menyediakan daftar semua

konfigurasi kamera yang tersedia melalui **Capture.list()**. Anda dapat menampilkannya di konsol pesan Anda, misalnya dengan menuliskan:

```
printArray(Capture.list());
```

Saat mencetak sebuah *array* ke konsol, menggunakan **printArray()** daripada **println()** akan memformat *array* tersebut dengan pemisah baris (*line breaks*) dan indeks numerik. Anda dapat menggunakan teks dari konfigurasi-konfigurasi tersebut untuk membuat sebuah objek **Capture**. Pada komputer Mac dengan kamera bawaan, misalnya, hal ini dapat terlihat seperti berikut:

```
video = new Capture(this, "name=FaceTime HD Camera (Built  
in),size=320x240,fps=30");
```

Capture.list() sebenarnya memberikan Anda sebuah *array*, jadi Anda juga dapat merujuk langsung pada indeks konfigurasi yang Anda inginkan.

```
video = new Capture(this, Capture.list()[0]);
```

Langkah 4. Mulai proses *capture*.

Setelah kamera siap, terserah Anda untuk memberi tahu Processing agar mulai menangkap (*capturing*) gambar.

```
void setup() {  
    video = new Capture(this, 320, 240);  
    video.start();  
}
```

Dalam hampir semua kasus, Anda ingin mulai melakukan *capture* langsung di dalam **setup()**. Meskipun demikian, **start()** adalah metodenya sendiri, dan Anda memiliki opsi, misalnya, untuk tidak memulai *capture* sampai waktu tertentu (seperti ketika sebuah tombol ditekan, dan sebagainya).

Langkah 5. Membaca gambar dari kamera.

Ada dua strategi untuk membaca *frame* dari kamera. Saya akan membahas keduanya secara singkat dan memilih salah satu untuk digunakan pada contoh-contoh selanjutnya dalam bab ini. Namun, kedua strategi tersebut bekerja berdasarkan prinsip dasar yang sama:

Saya hanya ingin membaca gambar dari kamera ketika sebuah *frame* baru tersedia untuk dibaca.

Untuk memeriksa apakah sebuah gambar tersedia, Anda menggunakan fungsi **available()**, yang mengembalikan nilai *true* atau *false* tergantung apakah ada data yang siap dibaca. Jika data tersedia, fungsi **read()** dipanggil, dan *frame* dari kamera dibaca ke dalam memori. Anda dapat melakukan ini berulang kali di dalam loop **draw()**, selalu memeriksa apakah ada gambar baru yang siap dibaca.

```
void draw() {  
    if (video.available()) {
```

```

        video.read();
    }
}

```

Strategi kedua, yaitu pendekatan “**event**”, membutuhkan sebuah fungsi yang dijalankan setiap kali suatu *event* terjadi — dalam hal ini adalah *event* kamera. Jika Anda ingat dari Bab 3, fungsi **mousePressed()** dieksekusi setiap kali mouse ditekan.

Dengan video, Anda memiliki opsi untuk mengimplementasikan fungsi **captureEvent()**, yang dipanggil setiap kali sebuah *capture event* terjadi, yaitu ketika sebuah *frame* baru tersedia dari kamera. Fungsi-fungsi *event* seperti **mousePressed()**, **keyPressed()**, **captureEvent()**, dan lainnya kadang disebut sebagai “**callback**”.

Sebagai tambahan, jika Anda mengikuti penjelasan sebelumnya dengan saksama, inilah bagian di mana *this* digunakan. Objek **Capture**, yaitu *video*, mengetahui cara memberi tahu *sketch* ini dengan memanggil **captureEvent()** karena Anda memberikan referensi *this* (*sketch* ini) ketika membuat objek **Capture** bernama *video*.

captureEvent() adalah sebuah fungsi dan karena itu harus berada dalam bloknnya sendiri, di luar **setup()** dan **draw()**.

```

void captureEvent(Capture video) {
    video.read();
}

```

Anda mungkin memperhatikan sesuatu yang aneh tentang **captureEvent()**. Fungsi tersebut memiliki sebuah argumen bertipe **Capture** dalam definisinya. Ini mungkin tampak redundan bagi Anda; lagipula, dalam contoh ini saya sudah memiliki variabel global **video**. Meskipun demikian, dalam kasus di mana Anda memiliki lebih dari satu perangkat *capture*, fungsi *event* yang sama dapat digunakan untuk semuanya, dan pustaka video akan memastikan bahwa objek **Capture** yang benar dilewatkan ke **captureEvent()**.

Untuk merangkum, saya ingin memanggil fungsi **read()** setiap kali ada sesuatu yang dapat dibaca, dan saya dapat melakukannya dengan dua cara:

1. **Memeriksa secara manual** menggunakan **available()** di dalam **draw()**, atau
2. **Membiarkan callback** menanganinya untuk Anda — yaitu melalui **captureEvent()**.

Untuk contoh-contoh dalam buku ini, saya akan menggunakan **captureEvent()**. Pendekatan ini membuat *sketch* berjalan lebih efisien dengan memisahkan logika pembacaan dari kamera dari loop animasi utama.

Langkah 6. Menampilkan gambar video.

Anda dapat menganggap objek **Capture** sebagai sebuah **PImage** yang berubah seiring waktu, dan memang, objek **Capture** dapat digunakan dengan cara yang sama persis seperti objek **PImage**.

```

image(video, 0, 0);

```

Semua ini digabungkan dalam **Contoh 12-1**.

Contoh 12-1. Menampilkan video

```

import processing.video.*; //Step 1: Import the video library!
Capture video; //Step 2: Declare a Capture object!

```

```

void captureEvent(Capture video) {
    video.read(); //Step 5: Read from the camera when a new image
    is available!
}

void setup() {
    size(320, 240);
    video = new Capture(this, 320, 240); //Step 3: Initialize
    Capture object!
    video.start(); //Step 4: Start the capturing process.
}

void draw() {
    image(video, 0, 0); //Step 6: Display the image.
}

```

Sekali lagi, apa pun yang dapat Anda lakukan dengan sebuah **PImage** (mengubah ukuran, memberi *tint*, memindahkan, dan sebagainya) juga dapat Anda lakukan dengan sebuah **Capture**. Selama Anda memanggil **read()** pada objek tersebut, gambar video akan terus diperbarui saat Anda memanipulasinya. Lihat **Contoh 12-2**.

Contoh 12-2. Memanipulasi gambar video

```

import processing.video.*;
Capture video;
void setup() {
    size(320, 240);
    video = new Capture(this, 320, 240);
    video.start();
}
void captureEvent(Capture video) {
    video.read();
}
void draw() {
    background(255);
    // A video image can be tinted just like a PImage.
    // It can also be moved, rotated, and sized just like a PImage.
    tint(mouseX, mouseY, 255);
    translate(width/2, height/2);
    imageMode(CENTER);
    rotate(PI/4);
    image(video, 0, 0, mouseX, mouseY);
}

```

Setiap contoh gambar dapat dibuat ulang menggunakan video. Berikut ini adalah contoh “mengatur kecerahan” menggunakan gambar video.

Contoh 12-3. Mengatur Kecerahan Video

```

// Step 1. Import the video library
import processing.video.*;
// Step 2. Declare a Capture object
Capture video;
void setup() {
    size(320, 240);
    // Step 3. Initialize Capture object via Constructor

```

```

video = new Capture(this, 320, 240);
video.start();
}
// An event for when a new frame is available
void captureEvent(Capture video) {
  // Step 4. Read the image from the camera.
  video.read();
}
void draw() {
  loadPixels();
  video.loadPixels();
  for (int x = 0; x < video.width; x++) {
    for (int y = 0; y < video.height; y++) {
      // Calculate the 1D location from a 2D grid
      int loc = x + y * video.width;
      // Get the red, green, blue values from a pixel
      float r = red (video.pixels[loc]);
      float g = green(video.pixels[loc]);
      float b = blue (video.pixels[loc]);
      // Calculate an amount to change brightness based on proximity
      to the mouse
      float d = dist(x, y, mouseX, mouseY);
      float adjustbrightness = map(d, 0, 100, 4, 0);
      r *= adjustbrightness;
      g *= adjustbrightness;
      b *= adjustbrightness;
      // Constrain RGB to make sure they are within 0-255 color range
      r = constrain(r, 0, 255);
      g = constrain(g, 0, 255);
      b = constrain(b, 0, 255);
      // Make a new color and set pixel in the window
      color c = color(r, g, b);
      pixels[loc] = c;
    }
  }
  updatePixels();
}

```

12.2 Recorded Video

Menampilkan video yang direkam mengikuti struktur yang hampir sama dengan video langsung. Pustaka video Processing menerima sebagian besar format file video; untuk detailnya, kunjungi referensi **Movie**

(<https://www.processing.org/reference/libraries/video/Movie.html>).

Langkah 1. Alih-alih menggunakan objek Capture, deklarasikan sebuah objek Movie.

```
Movie movie;
```

Langkah 2. Inisialisasikan objek *Movie*.

```
movie = new Movie(this, "testmovie.mov");
```

Satu-satunya argumen yang diperlukan adalah **this** dan **nama file video** yang dituliskan di dalam tanda kutip. File video tersebut harus disimpan di dalam direktori **data** milik *sketch*.

Langkah 3. Mulai memutar video.

Ada dua opsi: **play()**, yang memutar video sekali, atau **loop()**, yang memutarnya secara terus-menerus.

```
movie.loop();
```

Langkah 4. Membaca frame dari video.

Sekali lagi, langkah ini identik dengan *capture*. Anda bisa memeriksa apakah ada *frame* baru yang tersedia, atau menggunakan fungsi *callback*.

```
void draw() {  
    if (movie.available())  
    {  
        movie.read();  
    }  
}
```

Atau:

```
void movieEvent(Movie movie) {  
    movie.read();  
}
```

Langkah 5. Tampilkan video (movie).

```
image(movie, 0, 0);
```

Contoh 12-4 menunjukkan program yang sudah digabungkan seluruhnya.

Contoh 12-4 Menampilkan video yang direkam

```
import processing.video.*;  
Movie movie; //Step 1: Declare a Movie object!  
void setup() {  
    size(320, 240);  
    // Step 2: Initialize Movie object!  
    // The file "testmovie.mov" should live in the data folder.  
    movie = new Movie(this, "testmovie.mov");  
    // Step 3: Start playing movie.  
    // To play just once play() can be used instead.  
    movie.loop();  
}  
  
void movieEvent(Movie movie) {  
    movie.read(); // Step 4: Read new frames from the movie.  
}
```



```
void draw() {  
    image(movie, 0, 0); // Step 5: Display movie.  
}
```

Meskipun Processing bukanlah lingkungan yang paling canggih untuk menampilkan dan memanipulasi video yang direkam, terdapat beberapa fitur tingkat lanjut yang tersedia di dalam pustaka video. Ada fungsi-fungsi untuk mendapatkan **durasi** (panjang video dalam detik), untuk **mempercepat** atau **memperlambat** pemutaran, serta untuk **melompat ke titik tertentu** dalam video (di antara fitur lainnya). Jika Anda merasa performanya lambat dan pemutaran video terlihat tersendat-sendat, saya sarankan mencoba **renderer P2D** atau **P3D**. Berikut ini adalah contoh yang menggunakan **jump()** (melompat ke titik tertentu dalam video) dan **duration()** (mengembalikan panjang video dalam detik).

Contoh 12-5. Menggeser (scrubbing) maju dan mundur pada video

```
import processing.video.*;  
Movie movie;  
void setup() {  
    size(1024, 720);  
    background(0);  
    movie = new Movie(this, "testmovie.mov");  
}  
void movieEvent(Movie movie) {  
    movie.read();  
}  
void draw() {  
    // Ratio of mouse X over width  
    float ratio = mouseX / (float) width;  
    movie.jump(ratio * movie.duration());  
    image(movie, 0, 0);  
}
```

Dalam contoh ini, jika **mouseX** bernilai **0**, video akan melompat ke **awal**. Jika **mouseX** bernilai **width**, video akan melompat ke **akhir**. Nilai lainnya akan berada **di antara keduanya**!

Fungsi **jump()** memungkinkan Anda melompat secara langsung ke titik waktu tertentu di dalam video. Sementara itu, **duration()** mengembalikan total durasi video dalam satuan detik.

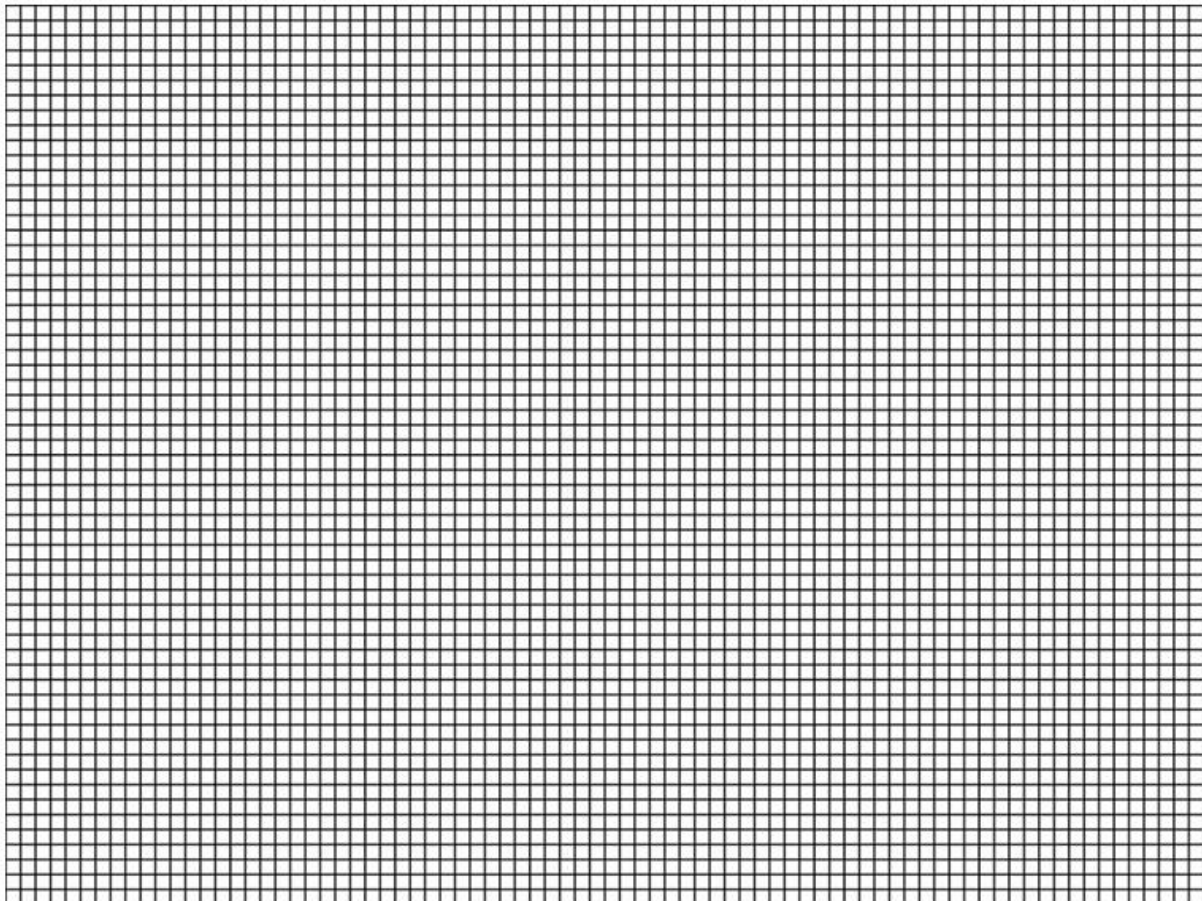
12.3 Software Mirror

Dengan semakin banyaknya kamera video kecil yang terpasang pada komputer pribadi, pengembangan perangkat lunak yang memanipulasi gambar secara **real-time** menjadi semakin populer. Aplikasi jenis ini kadang disebut sebagai **“mirror” (cermin digital)**, karena memberikan pantulan digital dari citra pengguna. Kumpulan fungsi grafis Processing yang sangat luas serta kemampuannya menangkap video secara real-time menjadikannya lingkungan yang sangat baik untuk membuat prototipe dan bereksperimen dengan *software mirror*.

Anda dapat menerapkan teknik pemrosesan citra dasar pada gambar video, yaitu dengan membaca dan mengganti piksel satu per satu. Melangkah lebih jauh, Anda dapat membaca warna piksel dan menerapkannya pada bentuk-bentuk yang digambar di layar.

Saya akan memulai dengan sebuah contoh yang menangkap video pada resolusi **80 × 60 piksel** dan merendernya pada jendela berukuran **640 × 480**. Untuk setiap piksel dalam video tersebut, saya akan menggambar sebuah persegi panjang berukuran **8 × 8 piksel**.

Mari kita tulis terlebih dahulu program yang menampilkan grid persegi panjang tersebut. Pada contoh berikut, variabel **videoScale** menyimpan rasio ukuran piksel jendela terhadap ukuran grid, dan untuk setiap kolom serta baris, sebuah persegi panjang digambar pada lokasi **(x, y)** yang telah diskalakan ukurannya menggunakan *videoScale*.



Contoh 12-6 Menggambar Grid 8 × 8 Persegi

```
// Size of each cell in the grid, ratio of window size to video size
int videoScale = 8;
// Number of columns and rows in the system
int cols, rows;
void setup() {
  size(640, 480);
  // Initialize columns and rows
  cols = width/videoScale;
  rows = height/videoScale;
}
void draw() {
  // Begin loop for columns
  for (int i = 0; i < cols; i++) {
    // Begin loop for rows
    for (int j = 0; j < rows; j++) {
```

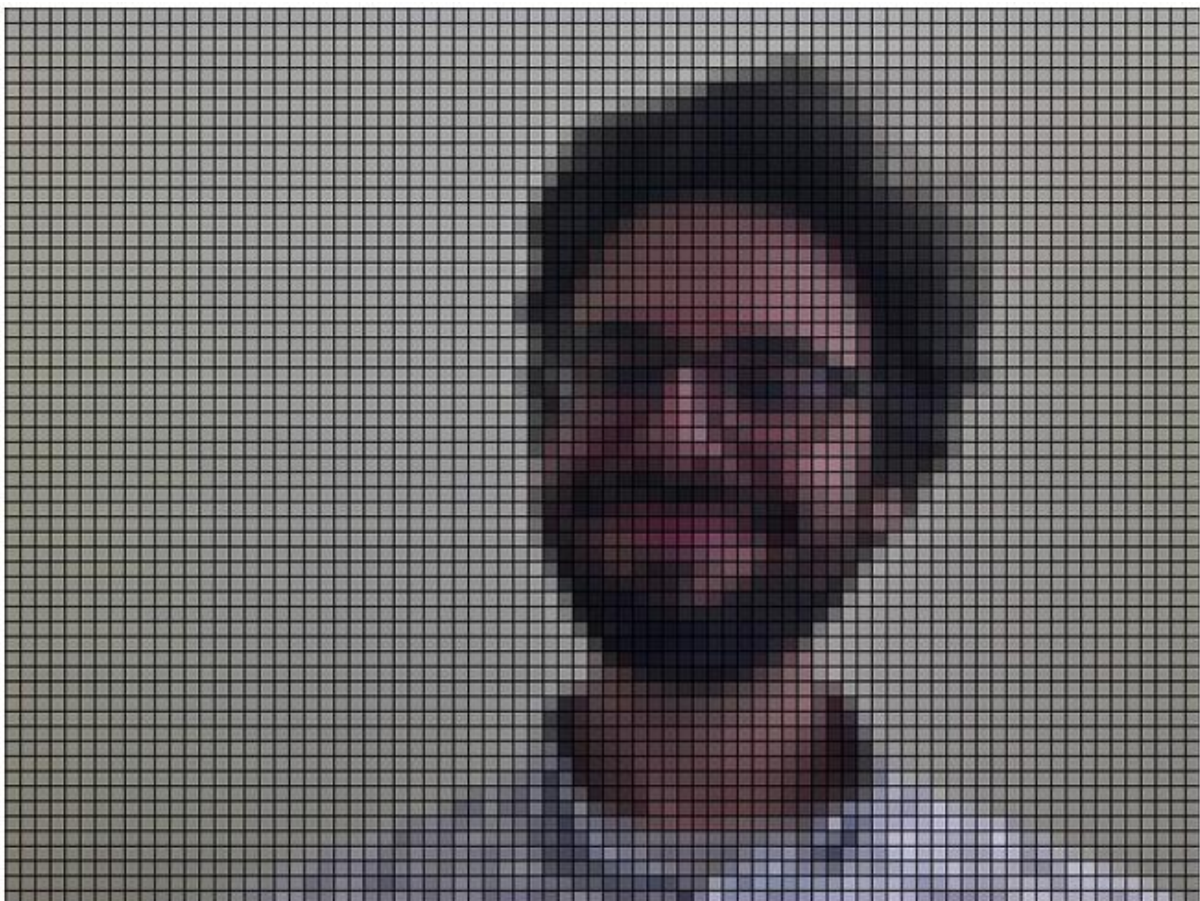
```
// Scaling up to draw a rectangle at (x,y)
int x = i*videoScale;
int y = j*videoScale;
fill(255);
stroke(0);
rect(x, y, videoScale, videoScale);
}
}
```

Mengetahui bahwa saya ingin memiliki kotak berukuran delapan piksel lebar dan delapan piksel tinggi, saya dapat menghitung jumlah kolom sebagai lebar dibagi delapan dan jumlah baris sebagai tinggi dibagi delapan.

$640/8 = 80$ kolom

$480/8 = 60$ baris

Sekarang saya dapat menangkap gambar video dengan ukuran **80×60** . Ini berguna karena menangkap video **640×480** dari kamera bisa lebih lambat dibandingkan **80×60** . Saya hanya ingin menangkap informasi warna pada resolusi yang dibutuhkan untuk sketsa.



Untuk setiap kotak pada kolom i dan baris j , saya mencari warna pada piksel (i, j) dalam gambar video dan mewarnainya sesuai. Lihat contoh berikut dengan bagian-bagian baru yang dicetak **tebal**:

Contoh 12-7. Video pixelation

```
import processing.video.*;

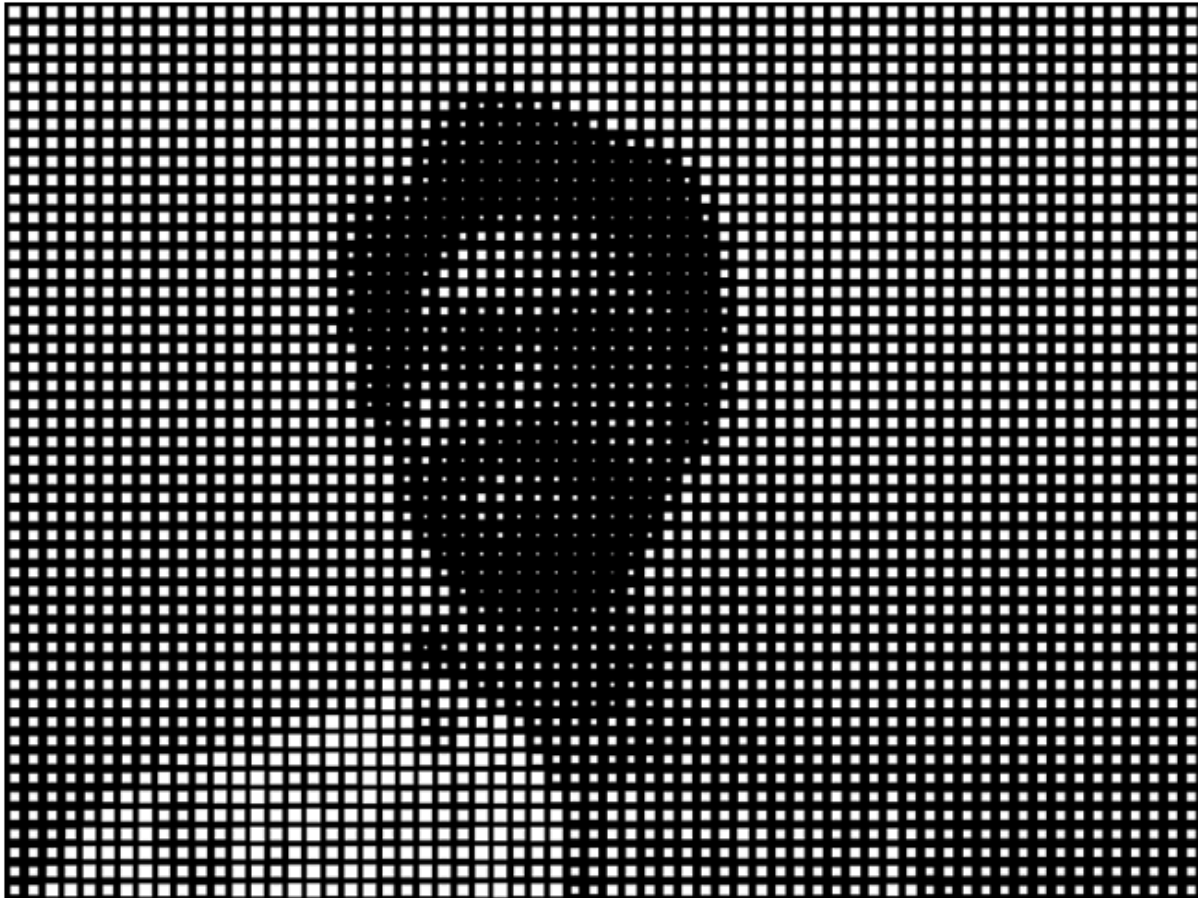
// Size of each cell in the grid, ratio of window size to video size
int videoScale = 8;
// Number of columns and rows in the system
int cols, rows;
// Variable to hold onto Capture object
Capture video;

void setup() {
  size(640, 480);
  // Initialize columns and rows
  cols = width/videoScale;
  rows = height/videoScale;
  background(0);
  video = new Capture(this, cols, rows);
  video.start();
}

// Read image from the camera
void captureEvent(Capture video) {
  video.read();
}

void draw() {
  video.loadPixels();
  // Begin loop for columns
  for (int i = 0; i < cols; i++) {
    // Begin loop for rows
    for (int j = 0; j < rows; j++) {
      // Where are you, pixel-wise?
      int x = i*videoScale;
      int y = j*videoScale;
      color c = video.pixels[i + j*video.width];
      fill(c);
      stroke(0);
      rect(x, y, videoScale, videoScale);
    }
  }
}
```

Seperti yang dapat Anda lihat, memperluas sistem grid sederhana untuk memasukkan warna dari video hanya memerlukan beberapa tambahan. Saya harus mendeklarasikan dan menginisialisasi objek **Capture**, membacanya, dan mengambil warna dari array piksel. Pemetaan warna piksel ke bentuk dalam grid yang lebih bebas (tidak literal) juga bisa diterapkan. Dalam contoh berikut, hanya warna hitam dan putih yang digunakan. Kotak-kotak lebih besar di tempat piksel yang lebih terang muncul dalam video, dan lebih kecil untuk piksel yang lebih gelap.

**Contoh 12-8 Brightness mirror**

```
// Each pixel from the video source is drawn as
// a rectangle with size based on brightness.

import processing.video.*;
// Size of each cell in the grid
int videoScale = 10;
// Number of columns and rows in the system
int cols, rows;
// Variable for capture device
Capture video;

void setup() {
  size(640, 480);
  // Initialize columns and rows
  cols = width / videoScale;
  rows = height / videoScale;
  // Construct the Capture object
  video = new Capture(this, cols, rows);
  video.start();
}

void captureEvent(Capture video) {
  video.read();
}
```

```

void draw() {
  background(0);
  video.loadPixels();
  // Begin loop for columns
  for (int i = 0; i < cols; i++) {
    // Begin loop for rows
    for (int j = 0; j < rows; j++) {
      // Where are you, pixel-wise?
      int x = i*videoScale;
      int y = j*videoScale;

      // Reverse the column to mirror the image.
      int loc = (video.width - i - 1) + j * video.width;

      color c = video.pixels[loc];
      // A rectangle's size is calculated as a function of the
      pixel's brightness.
      // A bright pixel is a large rectangle, and a dark pixel is a
      small one.
      float sz = (brightness(c)/255) * videoScale;

      rectMode(CENTER);
      fill(255);
      noStroke();
      rect(x + videoScale/2, y + videoScale/2, sz, sz);
    }
  }
}

```

Seringkali berguna untuk memikirkan pengembangan **cermin perangkat lunak** dalam dua langkah. Ini juga akan membantu Anda berpikir di luar pemetaan piksel ke bentuk pada grid yang lebih jelas.

Langkah 1. Kembangkan pola menarik yang menutupi seluruh jendela.

Langkah 2. Gunakan piksel dari video sebagai **look-up table** untuk mewarnai pola tersebut. Misalnya untuk Langkah 1, saya menulis program yang menggores garis acak di sekitar jendela. Berikut algoritmanya, ditulis dalam **pseudocode**:

1. Mulai dengan posisi (x,y) di tengah layar.
2. Ulangi langkah-langkah berikut selamanya:
 - Pilih (x,y) baru, tetap berada di dalam jendela.
 - Gambar garis dari (x,y) lama ke (x,y) baru.
 - Simpan (x,y) baru sebagai posisi lama berikutnya.

Contoh 12-9. The scribbler

```

// Two global variables
float x;
float y;

void setup() {
  size(640, 480);
  background(255);
  // Start x and y in the center
  x = width/2;

```

```

    y = height/2;
}

void draw() {
    float newx = constrain(x + random(-20, 20), 0, width);
    float newy = constrain(y + random(-20, 20), 0, height);

    // Line from (x,y) to the (newx,newy)
    stroke(0);
    strokeWeight(4);
    line(x, y, newx, newy);

    x = newx;
    y = newy;
}

```



Sekarang setelah saya menyelesaikan sketsa pembuat pola, saya bisa mengubah **stroke()** untuk menetapkan warna sesuai dengan gambar video. Perhatikan lagi baris-baris kode baru yang ditambahkan **dalam huruf tebal** pada kode berikut:

Contoh 12-10 The Scribbler Mirror

```

import processing.video.*;
// Two global variables
float x;
float y;

// Variable to hold onto Capture object.
Capture video;

void setup() {
    size(640, 480);
    background(255);
    // Start x and y in the center
    x = width/2;
    y = height/2;
    // Start the capture process
    video = new Capture(this, width, height);
    video.start();
}

```

```
}

void captureEvent(Capture video) {
    // Read image from the camera
    video.read();
}

void draw() {
    video.loadPixels();
    float newx = constrain(x + random(-20, 20), 0, width);
    float newy = constrain(y + random(-20, 20), 0, height-1);

    // Find the midpoint of the line
    int midx = int((newx + x) / 2);
    int midy = int((newy + y) / 2);
    // Pick the color from the video, reversing x
    color c = video.pixels[(width-1-midx) + midy*video.width];

    // Draw a line from (x,y) to (newx,newy)
    stroke(c);
    strokeWeight(4);
    line(x, y, newx, newy);

    // Save (newx,newy) in (x,y)
    x = newx;
    y = newy;
}
```

Jika jendela lebih besar (misalnya 800 × 600 piksel), Anda mungkin ingin **mengecilkan gambar yang ditangkap** sehingga tidak perlu menangkap gambar dengan resolusi yang sangat tinggi.



12.4 Video as sensor, computer vision

Setiap contoh dalam bab ini telah memperlakukan kamera video sebagai sumber data untuk citra digital yang ditampilkan di layar. Bagian ini akan memberikan pengenalan sederhana tentang hal-hal yang dapat Anda lakukan dengan kamera video **tanpa menampilkan gambarnya**, yaitu “**computer vision**”. **computer vision** adalah bidang penelitian ilmiah yang didedikasikan untuk mesin yang dapat “melihat”, menggunakan kamera sebagai sensor.

Untuk lebih memahami cara kerja algoritma penglihatan komputer, saya akan menulis semua kode **pada tingkat piksel demi piksel**. Namun, untuk mengeksplorasi topik ini lebih lanjut, Anda bisa mempertimbangkan untuk mengunduh beberapa **pihak ketiga computer vision libraries** yang tersedia untuk Processing. Banyak dari perpustakaan ini memiliki fitur canggih yang lebih luas daripada apa yang akan dibahas dalam bab ini. Gambaran singkat tentang perpustakaan-perpustakaan tersebut akan disajikan di akhir bagian ini.

Mari kita mulai dengan contoh sederhana.

Kamera video adalah teman Anda karena memberikan banyak informasi. Gambar berukuran 320×240 piksel memiliki **76.800 piksel**! Bagaimana jika semua piksel itu diringkas menjadi satu angka: **kecerahan keseluruhan ruangan**? Hal ini bisa dilakukan dengan sensor cahaya seharga satu dolar (atau “photocell”), tetapi sebagai latihan, saya akan menggunakan webcam untuk melakukannya.

Anda telah melihat dalam contoh-contoh sebelumnya bahwa nilai kecerahan dari sebuah piksel dapat diambil dengan fungsi **brightness()**, yang mengembalikan angka **floating point** antara 0 dan 255. Baris kode berikut mengambil nilai kecerahan untuk piksel pertama dalam gambar video.

```
float brightness = brightness(video.pixels[0]);
```

Saya kemudian dapat menghitung **kecerahan keseluruhan** (yaitu, rata-rata) dengan menjumlahkan semua nilai kecerahan dan membaginya dengan **total jumlah piksel**.

```
video.loadPixels();
// Start with a total of 0
float totalBrightness = 0;
// Sum the brightness of each pixel
for (int i = 0; i < video.pixels.length; i++) {
  color c = video.pixels[i];
  totalBrightness += brightness(c); // Sum all brightness values.
}

// Compute the average
// Average brightness = total brightness / total pixels
float averageBrightness = totalBrightness / video.pixels.length;
// Display the background as average brightness
background(averageBrightness);
```

Sebelum Anda terlalu antusias merayakan pencapaian ini, perlu diingat bahwa meskipun contoh ini merupakan demonstrasi yang sangat baik dari algoritma yang menganalisis data dari sumber video, hal itu **belum memanfaatkan sepenuhnya kekuatan apa yang bisa “dilihat” dengan kamera video**. Bagaimanapun, gambar video bukan hanya sekadar kumpulan warna, tetapi juga **kumpulan warna yang terorientasi secara spasial**. Dengan mengembangkan algoritma yang menelusuri piksel dan mengenali pola, Anda bisa mulai membuat aplikasi **computer vision** yang lebih canggih.

Melacak warna paling terang adalah langkah pertama yang bagus. Bayangkan sebuah ruangan gelap dengan satu sumber cahaya yang bergerak. Dengan teknik yang akan Anda

pelajari, sumber cahaya itu bisa menggantikan mouse sebagai bentuk interaksi. Ya, Anda sedang berada di jalur untuk bermain **Pong dengan senter**.

Pertama, saya akan meneliti bagaimana menelusuri sebuah gambar dan menemukan lokasi (x,y) dari **piksel paling terang**. Strategi yang akan saya gunakan adalah **mengulang semua piksel**, mencari piksel “terang dunia” (menggunakan fungsi **brightness()**). Awalnya, “rekor dunia” akan dipegang oleh piksel pertama. Saat piksel lain mengalahkan rekor itu, mereka akan menjadi pemegang rekor baru. Di akhir loop, piksel yang memegang rekor saat itu akan mendapatkan penghargaan “**Piksel Paling Terang di Gambar**”.

Berikut kodenya:

```
// The world record is 0 at first
float worldRecord = 0.0;
// Which pixel will win the prize?
int xRecordHolder = 0;
int yRecordHolder = 0;
for (int x = 0; x < video.width; x++) {
  for (int y = 0; y < video.height; y++) {
    // What is current brightness
    int loc = x + y*video.width;
    float currentBrightness = brightness(video.pixels[loc]);
    if (currentBrightness > worldRecord) {
      // Set a new record
      worldRecord = currentBrightness;
      // This pixel holds the record!
      xRecordHolder = x;
      yRecordHolder = y;
    }
  }
}
```

Perluasan alami dari contoh ini adalah **melacak warna tertentu**, bukan hanya warna paling terang.

Misalnya, saya bisa mencari piksel yang paling “merah” atau paling “biru” dalam sebuah gambar video. Untuk melakukan jenis analisis ini, saya perlu mengembangkan **metodologi untuk membandingkan warna**. Mari kita buat dua warna, **c1** dan **c2**.

```
color c1 = color(255, 100, 50);
color c2 = color(150, 255, 0);
```

Warna hanya dapat dibandingkan berdasarkan **komponen merah, hijau, dan biru**-nya, jadi saya harus terlebih dahulu **memisahkan nilai-nilai ini**.

```
float r1 = red(c1);
float g1 = green(c1);
float b1 = blue(c1);
float r2 = red(c2);
float g2 = green(c2);
```

```
float b2 = blue(c2);
```

Sekarang, saya siap untuk **membandingkan warna**. Salah satu strateginya adalah dengan **menjumlahkan nilai absolut dari perbedaan masing-masing komponen**. Memang terdengar rumit, tetapi sebenarnya cukup sederhana.

Ambil **r1 dikurangi r2**. Karena saya hanya peduli pada **besar perbedaannya**, bukan apakah nilainya positif atau negatif, ambil **nilai absolut** (versi positif dari angka tersebut). Lakukan hal yang sama untuk **hijau** dan **biru**, lalu jumlahkan semuanya.

```
float diff = abs(r1 - r2) + abs(g1 - g2) + abs(b1 - b2);
```

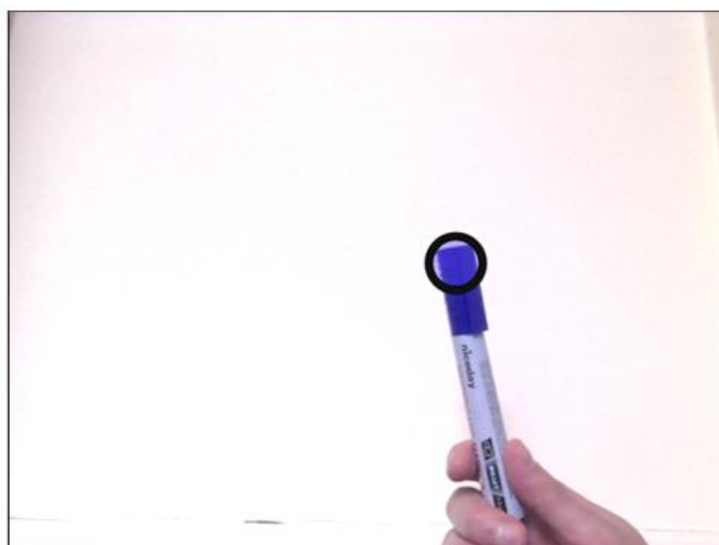
Meskipun cara ini **cukup memadai** (dan perhitungannya juga cepat), cara yang lebih akurat untuk menghitung perbedaan antar warna adalah dengan mengambil **“jarak” antara warna**. Baik, mungkin Anda berpikir: “Serius? Bagaimana mungkin sebuah warna bisa jauh atau dekat dengan warna lain?”

Nah, Anda tahu bahwa jarak antara dua titik dihitung menggunakan **Teorema Pythagoras**. Anda bisa menganggap warna sebagai titik dalam **ruang tiga dimensi**, hanya saja alih-alih (x,y,z), Anda memiliki (r,g,b). Jika dua warna berada **dekat satu sama lain** dalam ruang warna ini, artinya mereka **mirip**; jika **jauh**, artinya mereka **berbeda**.

```
float diff = dist(r1, g1, b1, r2, g2, b2);
```

Mencari **piksel paling merah** dalam sebuah gambar, misalnya, berarti mencari warna yang **paling mendekati merah** — (255,0,0).

Dengan menyesuaikan kode pelacakan kecerahan untuk mencari **piksel yang paling mendekati warna tertentu** (alih-alih yang paling terang), saya bisa membuat **sketsa pelacakan warna**. Dalam contoh berikut, pengguna dapat **mengklik mouse pada sebuah warna di gambar** untuk dilacak. Sebuah lingkaran hitam akan muncul di lokasi yang **paling sesuai dengan warna tersebut**. Lihat Gambar berikut.



Contoh 12-11. Simple color tracking

```
import processing.video.*;
// Variable for capture device
Capture video;
color trackColor;

void setup() {
  size(320, 240);
  video = new Capture(this, width, height);
  video.start();
  // Start off tracking for red
  trackColor = color(255, 0, 0);
}

void captureEvent(Capture video) {
  // Read image from the camera
  video.read();
}

void draw() {
  video.loadPixels();
  image(video, 0, 0);
  float worldRecord = 500;
  // (x,y) coordinate of closest color
  int closestX = 0;
  int closestY = 0;
  // Begin loop to walk through every pixel
  for (int x = 0; x < video.width; x++) {
    for (int y = 0; y < video.height; y++) {
      int loc = x + y * video.width;
      // What is current color
      color currentColor = video.pixels[loc];
      float r1 = red(currentColor);
      float g1 = green(currentColor);
      float b1 = blue(currentColor);
      float r2 = red(trackColor);
      float g2 = green(trackColor);
      float b2 = blue(trackColor);
      // Using euclidean distance to compare colors
      float d = dist(r1, g1, b1, r2, g2, b2);
      // If current color is more similar to tracked
      // color than closest color, save current location
      // and current difference
      if (d < worldRecord) {
        worldRecord = d;
        closestX = x;
        closestY = y;
      }
    }
  }

  if (worldRecord < 10) {
    // Draw a circle at the tracked pixel
    fill(trackColor);
    strokeWeight(4);
```

```

stroke(0);
ellipse(closestX, closestY, 16, 16);
}
}

void mousePressed() {
// Save color where the mouse is clicked in trackColor variable
int loc = mouseX + mouseY * video.width;
trackColor = video.pixels[loc];
}

```

12.5 Background removal

Perbandingan jarak untuk warna juga terbukti **berguna dalam algoritma computer vision lainnya**, seperti **penghapusan latar belakang**. Misalnya, Anda ingin menampilkan video saat Anda menari hula, tetapi Anda tidak ingin menari di kantor Anda saat ini, melainkan di pantai dengan ombak di belakang Anda.

Penghapusan latar belakang adalah teknik yang memungkinkan Anda **menghapus latar belakang** dari sebuah gambar (kantor Anda) dan menggantinya dengan piksel apa pun yang diinginkan (pantai), sambil tetap menjaga **latar depan** (Anda yang menari) tetap utuh.

Berikut algoritmenya:

- **Mengingat gambar latar belakang.**
- Periksa setiap piksel dalam **frame video saat ini**. Jika piksel tersebut sangat berbeda dari piksel yang bersesuaian di gambar latar belakang, maka itu adalah **piksel latar depan**. Jika tidak, itu adalah **piksel latar belakang**. Tampilkan hanya **piksel latar depan**.

Untuk mendemonstrasikan algoritma di atas, mari kita buat **reverse green screen**. Sketsa ini akan **menghapus latar belakang dari sebuah gambar** dan menggantinya dengan piksel hijau.

Langkah pertama adalah “**mengingat**” **latar belakang**. Latar belakang pada dasarnya adalah **snapshot** dari video. Karena gambar video berubah seiring waktu, saya harus **menyimpan salinan sebuah frame video** dalam objek **PImage** terpisah.

```

PImage backgroundImage;
void setup() {
backgroundImage = createImage(video.width, video.height, RGB);
}

```

Saat **backgroundImage** dibuat, itu adalah gambar kosong dengan **dimensi yang sama** dengan video. Dalam bentuk ini, gambar tersebut **tidak terlalu berguna**, jadi saya perlu **menyalin gambar dari kamera ke backgroundImage** ketika saya ingin mengingat latar belakang. Mari lakukan ini **saat mouse ditekan**.

```

void mousePressed() {
// Copying the current frame of video into the backgroundImage object
// Note copy takes 5 arguments:
// The source image
// x, y, width, and height of region to be copied from the source

```

```
// x, y, width, and height of copy destination
backgroundImage.copy(video, 0, 0, video.width, video.height,
    0, 0, video.width, video.height);
backgroundImage.updatePixels();
}
```

Setelah saya menyimpan gambar latar belakang, saya bisa **menelusuri semua piksel** dalam frame saat ini dan **membandingkannya dengan latar belakang** menggunakan perhitungan jarak. Untuk setiap piksel (x,y), saya menggunakan kode berikut:

```
int loc = x + y * video.width; // Step 1: what is the 1D pixel location?
color fgColor = video.pixels[loc]; // Step 2: the foreground color
color bgColor = backgroundImage.pixels[loc]; // Step 3: the background
color

// Step 4: Compare the foreground and background color
float r1 = red(fgColor); float g1 = green(fgColor); float b1 =
blue(fgColor);
float r2 = red(bgColor); float g2 = green(bgColor); float b2 =
blue(bgColor);
float diff = dist(r1, g1, b1, r2, g2, b2);
// Step 5: Is the foreground color different from the background
color
if (diff > threshold) {
    // If so, display the foreground color
    pixels[loc] = fgColor;
} else {
    // If not, display green
    pixels[loc] = color(0, 255, 0);
}
```

Kode di atas mengasumsikan adanya variabel bernama **threshold**. Semakin rendah nilai threshold, semakin mudah sebuah piksel dianggap **sebagai latar depan**. Piksel tersebut tidak harus sangat berbeda dari piksel latar belakang.

Berikut contoh lengkapnya dengan **threshold** sebagai variabel global.

Contoh 12-12. Simple background removal

```
import processing.video.*;
// Variable for capture device
Capture video;
// Saved background
PImage backgroundImage;
// How different must a pixel be to be foreground
float threshold = 20;
void setup() {
    size(320, 240);
    video = new Capture(this, width, height);
    video.start();
    // Create an empty image the same size as the video
    backgroundImage = createImage(video.width, video.height, RGB);
}
```



```

}

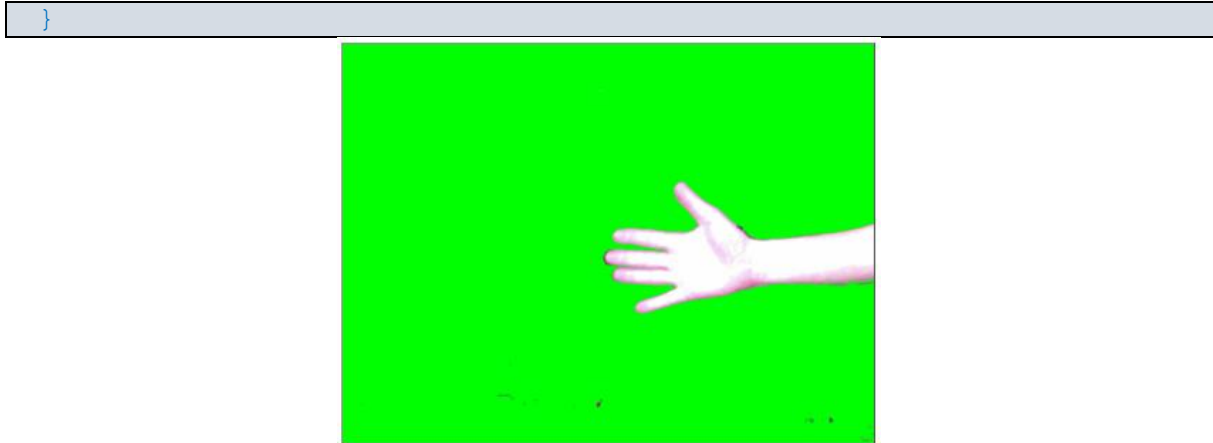
void captureEvent(Capture video) {
    video.read();
}

void draw() {
    loadPixels();
    video.loadPixels();
    backgroundImage.loadPixels();

    // Draw the video image on the background
    image(video, 0, 0);
    // Begin loop to walk through every pixel
    for (int x = 0; x < video.width; x++) {
        for (int y = 0; y < video.height; y++) {
            int loc = x + y*video.width; // Step 1, what is the 1D pixel
location
            color fgColor = video.pixels[loc]; // Step 2, what is the
foreground color
            // Step 3, what is the background color
            color bgColor = backgroundImage.pixels[loc];
            // Step 4, compare the foreground and background color
            float r1 = red (fgColor);
            float g1 = green(fgColor);
            float b1 = blue (fgColor);
            float r2 = red (bgColor);
            float g2 = green(bgColor);
            float b2 = blue (bgColor);
            float diff = dist(r1, g1, b1, r2, g2, b2);
            // Step 5, Is the foreground color different from the
background color
            if (diff > threshold) {
                // If so, display the foreground color
                pixels[loc] = fgColor;
            } else {
                // If not, display green
                pixels[loc] = color(0, 255, 0);
            }
        }
    }
    updatePixels();
}

void mousePressed() {
    // Copying the current frame of video into the backgroundImage
object
    // Note copy takes nine arguments:
    // The source image
    // x, y, width, and height of region to be copied from the source
    // x, y, width, and height of copy destination
    backgroundImage.copy(video, 0, 0, video.width, video.height,
        0, 0, video.width, video.height);
    backgroundImage.updatePixels();
}

```



Saat Anda menjalankan contoh ini, **keluarlah dari frame**, klik mouse untuk **mengingat latar belakang tanpa Anda di dalamnya**, lalu **masuk kembali ke frame**; Anda akan melihat hasilnya seperti yang ditunjukkan pada Gambar di atas.

Jika sketsa ini **tidak berfungsi sama sekali** untuk Anda, periksa fitur “otomatis” apa saja yang diaktifkan pada kamera Anda. Misalnya, jika kamera diatur untuk **menyesuaikan kecerahan atau white balance secara otomatis**, ini akan menjadi masalah. Meskipun gambar latar belakang sudah diingat, begitu seluruh gambar menjadi lebih terang atau berubah warnanya, sketsa ini akan **mengira semua piksel telah berubah** dan oleh karena itu dianggap sebagai **latar depan**! Untuk hasil terbaik, **nonaktifkan semua fitur otomatis** pada kamera Anda.

12.6 Soal Latihan

1. Alih-alih mengganti latar belakang dengan piksel hijau, gantilah dengan gambar lain. Nilai berapa saja yang bekerja dengan baik untuk threshold, dan nilai berapa yang sama sekali tidak berfungsi? Cobalah mengontrol variabel threshold dengan menggunakan mouse.
2. Buatlah sebuah sketch yang mencari lokasi rata-rata dari gerakan (motion). Bisakah kamu membuat sebuah elips yang mengikuti gerakan tanganmu yang melambai?

Daftar Pustaka:

1. Shiffman, Daniel. 2015. *Learning Processing: A Beginner's Guide to Programming Images, Animation, and Interaction Second Edition*. United States of America: Morgan Kaufmann.
2. <http://www.processing.org/reference/libraries/video/Capture.html>